

Cyber Threats and Their Mitigations: A Study of Software Supply Chain Risks



Group 4

Adesh Palkar	23114004
Nisarg Prajapati	23114073
Utkarsh Kumar	23114101
Tanay Kapadia	23323044
Parth Baranwal	23114076

Group 4

Computer Science and Engineering

Indian Institute of Technology Roorkee

Roorkee - 247667 (INDIA)

Technical Communication Presentation Report

April, 2026

**©INDIAN INSTITUTE OF TECHNOLOGY ROORKEE, ROORKEE-2026
ALL RIGHTS RESERVED**

Statement of Contributions

This report, titled “Cyber Threats and Their Mitigations: A Study of Software Supply Chain Risks,” was prepared as a collaborative group submission. The contributions of each author are listed exactly below.

Author	Contribution
Adesh Palkar (23114004)	Concise overview of modern cybersecurity threats and their practical mitigations.
Nisarg Prajapati (23114073)	XZ Utils case study, attack-path analysis, problem statement, and broader impact discussion.
Utkarsh Kumar (23114101)	NDSS Research Paper: MALOSS pipeline, Key Findings, Mitigation Strategies, and Limitations.
Tanay Kapadia (23323044)	Dependency confusion: overview, origin, solution, and limitations.
Parth Baranwal (23114076)	Existing solutions and future solution possibilities.

TABLE 1: Author-wise statement of contributions for Group 4.

Chapter-wise “Declaration of Contributions” sections are also provided at the beginning of every chapter after Chapter 1, as required.

Abstract

Modern software systems rely heavily on third-party dependencies and package managers. This dependence has increased development speed, but it has also expanded the cyber attack surface, especially in software supply chains. This report studies major cyber threat classes and narrows focus to software supply chain attacks as a high-impact, ecosystem-level risk.

The report presents: (i) an overview of malware, phishing, denial-of-service, insider threats, and supply chain attacks; (ii) a case study of the 2024 XZ Utils backdoor incident; (iii) a review of related research, including MALOSS and dependency confusion analysis; and (iv) a comparative discussion of existing and proposed mitigations, including eBPF-based monitoring, static analysis, and symbolic execution assisted triage.

Our key conclusion is that trust itself becomes the primary attack vector in software supply chains. No single defense is sufficient; robust mitigation requires layered controls across maintainers, registries, developers, and deployment pipelines. The report closes with practical future directions: dependency provenance verification, stronger release integrity checks, continuous monitoring, and faster ecosystem-wide response to newly discovered malicious packages.

Acknowledgements

We sincerely thank the course instructor for guidance on technical communication, problem framing, and report organization. We also acknowledge the authors of the research papers, advisories, and incident reports used in this study, which helped us understand software supply chain attacks from both academic and practical viewpoints.

As a team, we appreciate the collaborative effort of all Group 4 members in preparing the literature review, case study analysis, mitigation discussion, and final documentation.

Date:

Group 4

Contents

Statement of Contributions	iii
1 Introduction	1
1.1 Modern Cyber Threat Landscape	1
1.2 Scope and Focus	1
1.3 Key Contributions of This Report	1
1.4 Report Organization	2
2 Preliminaries, Background, and Related Work	3
2.1 Preliminaries and Basic Notions	3
2.2 Threat Context for Supply Chains	3
2.3 Related Work	4
2.4 Why Existing Trust Models Fail	4
3 Case Study and Problem Statement	5
3.1 Case Study: XZ Utils Backdoor (2024)	5
3.2 Attack Path and Security Insight	5
3.3 Problem Statement	6
3.4 Why This Matters	6
4 In-depth Discussion of Existing and Proposed Solutions	7
4.1 Existing Solution Families	7
4.2 Dependency Confusion and Source-based Risk Assessment	8
4.3 Proposed Layered Mitigation Framework	8
4.4 Mitigation Limits and Practical Constraints	8
4.5 Chapter Summary	9
5 Conclusion and Future Work	10
5.1 Conclusion	10
5.2 Future Work	10

A	Supplementary Material and Source Snippets	11
A.1	Example Dependency Policy Script	11
A.2	Example CI Security Gate	12

Chapter 1

Introduction

Cybersecurity is no longer a niche technical concern. It directly affects daily life through banking, communication, healthcare, transportation, and digital public services. Modern attacks are targeted, persistent, and increasingly systemic because software systems are deeply connected through cloud services, CI/CD pipelines, open-source dependencies, and distributed teams.

This report presents a broad overview of contemporary cyber threats and then focuses on software supply chain attacks as the central theme. The focus is motivated by recent incidents where compromise of a trusted upstream component propagated silently to many downstream consumers.

1.1 Modern Cyber Threat Landscape

The most common operational threat classes discussed in this report are malware/ransomware, phishing and social engineering, denial-of-service (DDoS), insider threats, and software supply chain attacks. A concise summary is provided in [Table 1.1](#).

1.2 Scope and Focus

The broad threat overview is narrowed to supply chain attacks for three reasons:

- (i) the trust boundary in dependency ecosystems is often implicit,
- (ii) one upstream compromise can impact many organizations,
- (iii) detection and response are difficult when malicious behavior is hidden in normal update paths.

1.3 Key Contributions of This Report

The main contributions are:

TABLE 1.1: High-level threat classes and representative mitigations.

Threat Class	Typical Attack Vector	Representative Mitigations
Malware and Ransomware	Malicious payload execution for encryption, theft, or disruption	Endpoint protection, offline backups, rapid patching
Phishing and Social Engineering	Deceptive communication that steals credentials or installs malware	MFA, secure email gateway, user awareness
DDoS	Flooding network/service resources to exhaust availability	Traffic filtering, rate limiting, redundancy
Insider Threats	Misuse of legitimate internal access	Least privilege, monitoring, segregation of duties
Supply Chain Attack	Compromise of trusted package, dependency, or build path	Provenance checks, signed releases, continuous vetting

- a structured overview of modern cyber threats and mitigations;
- a focused case study of the XZ Utils incident (2024) and its attack path;
- synthesis of related work on package ecosystem security and dependency confusion;
- in-depth discussion of existing and proposed defenses, with explicit limitations and future directions.

1.4 Report Organization

Chapter 2 introduces background, core concepts, and related work. **Chapter 3** presents the XZ backdoor case and formal problem statement. **Chapter 4** provides in-depth discussion of existing and proposed solutions. **Chapter 5** summarizes conclusions and outlines future work.

Chapter 2

Preliminaries, Background, and Related Work

Declaration of Contributions

This chapter was prepared primarily by Utkarsh Kumar and Tanay Kapadia. Utkarsh Kumar synthesized related research on package ecosystem threats and the MALOSS pipeline, while Tanay Kapadia contributed dependency confusion background and static source analysis concepts. Adesh Palkar contributed the generalized threat taxonomy and chapter-level consolidation.

2.1 Preliminaries and Basic Notions

Modern applications depend on external libraries distributed through package registries such as npm, PyPI, and RubyGems. Let $G = (V, E)$ denote a dependency graph where each node $v \in V$ is a package and each edge $e \in E$ represents a dependency relation. A compromise in an upstream node can propagate along outgoing dependency paths to multiple downstream consumers.

This propagation model explains why software supply chain attacks are ecosystem risks rather than isolated endpoint events.

2.2 Threat Context for Supply Chains

Traditional cyber defenses are often perimeter-centric. In contrast, supply chain attacks exploit implicit trust in package origin, maintainer identity, build artifacts, and update channels. The attack vector is frequently trust itself: a dependency that appears legitimate can execute malicious logic during install, import, or runtime.

Conceptual Supply Chain Path: Developer publishes package → registry distributes package → downstream projects install dependency → compromise fans out to users. An attacker may inject code upstream or exploit naming/version resolution rules.

FIGURE 2.1: Conceptual propagation path in a software supply chain attack.

2.3 Related Work

Duan *et al.* presented one of the most influential ecosystem-scale studies on package manager attacks and proposed MALOSS (MALware detection for Open Source Software), combining metadata analysis, static analysis, dynamic analysis, and human validation [?].

Their findings show that malicious packages can persist for long periods and that attack vectors such as typosquatting and account compromise are recurrent. This supports a data-driven, continuous-analysis approach rather than one-time manual vetting.

Dependency confusion was later formalized as a practical and severe risk in mixed public-private dependency environments. Birsan demonstrated real-world exploitation in enterprise contexts [?], and Ding *et al.* proposed source tree matching for systematic risk assessment [?].

2.4 Why Existing Trust Models Fail

Most ecosystems optimize for package availability and developer convenience. As a result, they can inherit the following weaknesses:

- weak provenance guarantees for package origin and release artifacts;
- delayed detection of malicious behavior hidden in build logic;
- version-selection policies that can prioritize untrusted public packages.

These observations motivate the case study and problem formulation in [Chapter 3](#), followed by in-depth mitigation design in [Chapter 4](#).

Chapter 3

Case Study and Problem Statement

Declaration of Contributions

This chapter was prepared primarily by Nisarg Prajapati. Nisarg Prajapati developed the XZ Utils case narrative, attack-path analysis, and broader impact discussion. Adesh Palkar and Utkarsh Kumar assisted with threat framing and technical interpretation.

3.1 Case Study: XZ Utils Backdoor (2024)

XZ Utils is a widely used compression utility in Linux ecosystems and includes the `liblzma` library used by many software stacks. In 2024, malicious code associated with releases 5.6.0 and 5.6.1 raised serious concerns due to potential unauthorized access paths in environments linked with `sshd`/OpenSSH [? ?].

TABLE 3.1: Simplified timeline and implications of the XZ Utils incident.

Event		Observation
Upstream release activity		Malicious behavior was associated with release/build artifacts.
Potential integration path		Systems using affected dependency paths could inherit risk transitively.
Ecosystem response		Advisories and coordinated mitigation were issued to reduce spread.
Security lesson		A trusted low-level dependency can become a high-impact attack surface.

3.2 Attack Path and Security Insight

The attack path can be interpreted as follows:

1. compromise is introduced at an upstream package/release stage;

2. downstream systems consume the package through normal dependency workflows;
3. malicious behavior is activated in operational contexts where trust is assumed.

This pattern demonstrates a core insight: in supply chain attacks, *implicit trust* in dependency ecosystems can bypass traditional defensive boundaries.

3.3 Problem Statement

The widespread blind trust in unverified open-source dependencies exposes organizations to devastating software supply chain attacks.

Technically, this problem appears because:

- modern software depends on deep transitive dependency chains;
- package adoption is often based on popularity or convenience rather than verified provenance;
- release artifact integrity and maintainer-risk checks are inconsistently enforced.

3.4 Why This Matters

A single compromised package can affect many products and organizations across sectors. Attack impact propagates from upstream maintainers to downstream applications and then to end users. National-level advisories (for example, CISA guidance) reinforce the need for rapid, layered defense strategies and coordinated response [?].

The solution direction is developed in [Chapter 4](#), where we compare current controls and propose a composite mitigation strategy.

Chapter 4

In-depth Discussion of Existing and Proposed Solutions

Declaration of Contributions

This chapter was jointly prepared by Parth Baranwal, Utkarsh Kumar, and Tanay Kapadia. Parth Baranwal contributed the analysis of existing and future-oriented controls. Utkarsh Kumar contributed MALOSS and mitigation-limitations discussion. Tanay Kapadia contributed dependency confusion analysis and source-level detection approach.

4.1 Existing Solution Families

Current defenses against supply chain compromise can be grouped into four practical families:

1. **Runtime visibility and enforcement:** eBPF-based telemetry and policy enforcement at kernel level can improve detection of anomalous process, file, and network behavior.
2. **Registry-side controls:** MFA for maintainers, typo detection, rapid advisories, and package vetting reduce malicious publication success.
3. **Build and release integrity:** artifact signing, provenance metadata, and reproducible build checks reduce tampering risk.
4. **Automated package analysis:** pipelines such as MALOSS combine metadata, static, dynamic, and manual verification stages [?].

4.2 Dependency Confusion and Source-based Risk Assessment

Dependency confusion exploits package name and version resolution across public and private registries. If resolver policies prefer an untrusted public package, an attacker can hijack a dependency path [?].

Ding *et al.* proposed static source tree matching to compare intended components with actual dependencies, enabling automated discrepancy detection even in obfuscated or injected settings [?].

4.3 Proposed Layered Mitigation Framework

Based on the case study and related work, we propose a layered framework that combines prevention, detection, and response.

Algorithm 1 Layered Screening for Third-party Dependencies

Require: Dependency set D , policy rules P , trusted source map T

Ensure: Classified set of dependencies with risk levels

- 1: **for all** $d \in D$ **do**
 - 2: Validate source and maintainer metadata against T
 - 3: Run static checks on install/build scripts and sensitive APIs
 - 4: Trigger sandboxed dynamic checks for high-risk candidates
 - 5: Assign risk score and enforce policy action (allow, quarantine, reject)
 - 6: **end for**
 - 7: Publish alerts and update trust rules from analyst feedback
-

4.4 Mitigation Limits and Practical Constraints

No single method is complete. Main constraints include:

- static analysis may miss runtime-triggered behavior or obfuscated logic;
- dynamic analysis depends on execution paths and may miss dormant payloads;
- full manual verification does not scale to very large package ecosystems;
- delayed ecosystem response can prolong exposure windows.

Hence, effective defense is a socio-technical process that combines automation, policy enforcement, and rapid incident communication.

4.5 Chapter Summary

The chapter demonstrates that practical risk reduction requires layered controls, continuous monitoring, and explicit trust verification throughout the software lifecycle. These insights lead to the consolidated conclusions and roadmap in **Chapter 5**.

Chapter 5

Conclusion and Future Work

Declaration of Contributions

This chapter was jointly prepared by Parth Baranwal and Adesh Palkar, with final review by all Group 4 members.

5.1 Conclusion

This report examined modern cyber threats and focused on software supply chain attacks as a high-impact, trust-centric threat model. Through the XZ Utils case study ([Chapter 3](#)) and related research synthesis ([Chapter 2](#)), we observed that a single compromised upstream dependency can propagate risk to many downstream systems.

The in-depth analysis ([Chapter 4](#)) shows that existing solutions improve visibility and control but remain individually insufficient. The central conclusion is that dependency trust must be continuously verified, not implicitly assumed.

5.2 Future Work

The following directions are recommended for improving practical defenses:

1. standardized provenance and signed artifact verification across package ecosystems;
2. stronger CI/CD policy gates for dependency updates and transitive risk checks;
3. scalable hybrid analysis (static + dynamic + symbolic) with low false-positive rates;
4. faster ecosystem-wide notification and coordinated mitigation for exposed dependents;
5. broader adoption of security-by-default package publishing and maintainer hardening.

These directions support a shift from reactive patching to proactive and continuous software supply chain security.

Appendix A

Supplementary Material and Source Snippets

This appendix provides supplementary technical material used in the report. It includes representative policy checks and automation snippets for dependency screening.

A.1 Example Dependency Policy Script

```
1 import json
2
3 def evaluate_dependency(meta):
4     risk = 0
5
6     if not meta.get("signed_release", False):
7         risk += 3
8     if meta.get("new_maintainer", False):
9         risk += 2
10    if meta.get("suspicious_install_script", False):
11        risk += 4
12    if meta.get("typosquat_score", 0) > 0.8:
13        risk += 3
14
15    if risk >= 7:
16        return "quarantine"
17    if risk >= 4:
18        return "manual_review"
19    return "allow"
20
21
22 if __name__ == "__main__":
23     sample = {
24         "signed_release": False,
25         "new_maintainer": True,
26         "suspicious_install_script": True,
```

```
27     "typosquat_score": 0.1,  
28 }  
29 print(evaluate_dependency(sample))
```

LISTING A.1: Illustrative policy checks for dependency metadata.

A.2 Example CI Security Gate

```
1  #!/usr/bin/env bash  
2  set -e  
3  
4  echo "Running dependency integrity checks..."  
5  python scripts/policy_check.py  
6  
7  echo "Running static source checks..."  
8  ./scripts/static_scan.sh  
9  
10 echo "Blocking release if risk score is high"  
11 ./scripts/enforce_threshold.sh
```

LISTING A.2: Illustrative CI gate for dependency updates.

Listings [A.1](#) and [A.2](#) are illustrative examples and can be extended with organization-specific controls.

List of Figures

2.1 Conceptual propagation path in a software supply chain attack.	4
--	---

List of Tables

1	Author-wise statement of contributions for Group 4.	iii
1.1	High-level threat classes and representative mitigations.	2
3.1	Simplified timeline and implications of the XZ Utils incident.	5

List of Abbreviations

APT	Advanced Persistent Threat
CI/CD	Continuous Integration / Continuous Deployment
CISA	Cybersecurity and Infrastructure Security Agency
CVE	Common Vulnerabilities and Exposures
DDoS	Distributed Denial of Service
eBPF	Extended Berkeley Packet Filter
MFA	Multi-Factor Authentication
MALOSS	Malware detection for Open Source Software
NVD	National Vulnerability Database
SBOM	Software Bill of Materials
SMT	Satisfiability Modulo Theories

List of Symbols

$G = (V, E)$	Dependency graph of package ecosystem	–
V	Set of package nodes	–
E	Set of dependency edges	–
d	A dependency in the project dependency set	–
P	Security policy rule set	–
T	Trusted source and provenance map	–
$\rho(d)$	Risk score assigned to dependency d	–

List of Mathematical Symbols

$d \in D$	Dependency d belongs to the complete dependency set D .
$G = (V, E)$	A dependency relation exists between two packages if an edge is present in E .
$\rho(d) \geq \tau$	A dependency is quarantined when its risk score exceeds threshold τ .